

Set Operations in Relational Algebra and SQL

1. Introduction

Definition

Set operations combine the results of two or more queries in relational databases. They operate on **relations** and follow set theory principles.

Common set operations:

- Union ($\cup$)
- Intersection ($\cap$)
- Difference ($-$)
- Cartesian Product ($\times$)
- Division ($\div$)

2. Union ($\cup$)

Rule

Returns all tuples from two relations without duplicates. Relations must be **union-compatible** (same attributes and domains).

Relational Algebra

$$R \cup S$$

SQL

```
SELECT * FROM R
UNION
SELECT * FROM S;
```

Example

R		S	
A	B	A	B
1	X	2	Y
2	Y	4	W
3	Z		

Result of $R \cup S$:

A	B
1	X
2	Y
3	Z
4	W

3. Intersection ($\cap$)

Rule

Returns tuples that are present in **both** relations.

Relational Algebra

$$R \cap S$$

SQL

```
SELECT * FROM R
INTERSECT
SELECT * FROM S;
```

Result Example

A	B
2	Y

4. Difference ($-$)

Rule

Returns tuples that are in R but not in S .

Relational Algebra

$$R - S$$

SQL

```
SELECT * FROM R
EXCEPT
SELECT * FROM S;
```

Result Example

A	B
1	X
3	Z

5. Cartesian Product ($\times$)

Rule

Combines every tuple of R with every tuple of S .

Relational Algebra

$$R \times S$$

SQL

```
SELECT * FROM R, S;
```

6. Division ($\div$)

Rule

Finds tuples in R that are related to **all** tuples in S .

Relational Algebra

$$R \div S$$

Used when a query involves "for all" conditions.

7.Examples

Consider the query to find the set of all courses taught in the Fall 2017 semester, the Spring 2018 semester, or both. This information is contained in the **section** relation.

To find the set of all courses taught in the Fall 2017 semester, we write (using **selection** and **projection**):

$$\pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2017}(\text{section}))$$

To find the set of all courses taught in the Spring 2018 semester:

$$\pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2018}(\text{section}))$$

Union Operation: Courses offered in Fall 2017 or Spring 2018

The set of all course IDs taught in either or both semesters is the union of these two sets:

$$\pi_{\text{course_id}}(\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2017}(\text{section})) \cup \pi_{\text{course_id}}(\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2018}(\text{section}))$$

Note: The result contains eight course IDs, even though there are only three distinct courses in Fall 2017 and six in Spring 2018. Since relations are sets, duplicates such as CS-101 appearing in both semesters occur only once in the result.

Important points for union operation to make sense:

1. The two input relations must have the **same number of attributes** (arity).
2. The corresponding attributes must have **compatible types**.

Intersection Operation: Courses offered in both Fall 2017 and Spring 2018

To find courses taught in both semesters, use the intersection operation:

$$\pi_{\text{course_id}}(\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2017}(\text{section})) \cap \pi_{\text{course_id}}(\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2018}(\text{section}))$$

Difference Operation: Courses offered in Fall 2017 but not in Spring 2018

To find courses taught in Fall 2017 but *not* in Spring 2018, use set difference:

$$\pi_{\text{course_id}}(\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2017}(\text{section})) - \pi_{\text{course_id}}(\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2018}(\text{section}))$$

8. Summary Table

Operation	Relational Algebra Symbol	SQL Keyword
Union	$\cup$	UNION
Intersection	$\cap$	INTERSECT
Difference	$-$	EXCEPT / MINUS
Cartesian Product	$\times$	CROSS JOIN
Division	$\div$	No direct SQL keyword